PEFormer: Exploring Byte-level Transformer for Raw-Binary Malware Detection

Thin Tharaphe Thein[1,a)] Yoshiaki Shiraishi[1,b)]

Abstract: Conventional static malware detection approaches typically rely on manual feature engineering or intermediate disassembly. Although existing end-to-end convolutional models have enabled automated representation learning directly from raw executables, their ability to capture non-contiguous control-flow structures and global semantic correlations remains limited. In this study, we introduce PEFormer, a byte-level Transformer architecture for malware detection from raw Windows Portable Executable (PE) binaries. We investigate whether hierarchical self-attention applied directly to non-disassembled binaries can learn deeper representations of malicious behaviors than CNN-based detectors without platform-specific preprocessing. To address the quadratic computational complexity of standard Transformers, the proposed model incorporates convolutional downsampling and a windowed attention, which together enable efficient processing of binaries up to 1 MB. Experiments on a verified dataset of 176,079 samples demonstrate that the proposed byte-level Transformer achieves detection performance comparable to MalConv (F1: 0.9865 vs. 0.9818) while generalizing substantially better to unseen obfuscated malware variants, correctly detecting 99.2% of obfuscated samples compared to MalConv's 50.0%.

Keywords: Malware Detection, Transformer, Byte-level Representation Learning

1. Introduction

The rapid development of technology has transformed many aspects of our society. At the same time, malicious software attacks have escalated in both frequency and sophistication. Cybercriminals frequently deploy malware to execute unauthorized operations and compromise system integrity. As noted by van Liebergen et al.[1], Windows Portable Executable (PE) formats (e.g., .exe and .dll) account for the majority of the files submitted by end users to VirusTotal[*1]. This highlights the critical need for an effective PE malware detector. Furthermore, malware is rapidly evolving through techniques such as obfuscation, polymorphism, and AI-assisted generation, which make conventional signature-based and rule-based detection ineffective against previously unseen variants.

To address these limitations, machine learning (ML) offers an adaptive and data-driven alternative. Unlike rule-based approaches, ML-based malware detectors learn malicious patterns directly from data via static and dynamic analysis. The effectiveness of these approaches has been demonstrated in prior studies[2][3][4]. Static analysis extracts structural features such as byte n-grams, opcodes, and file headers, without executing the file[2][4]. In contrast, dynamic analysis observes how a suspicious file actually behaves in a sandbox environment by monitoring runtime indicators such as API calls, registry modifications, and suspicious network activity[4][5]. Both approaches generate feature sets that can be fed to ML classifiers. In practice, effective malware analysis rarely relies on a single detector. Instead, it combines multiple detection mechanisms to optimize overall detection performance[6]. Nevertheless, static analysis remains the first line of defense that a malicious payload must bypass.

Although manual feature engineering combined with gradient boosting models or Deep Neural Networks (DNNs) has demonstrated competitive malware detection performance[2][3], it relies on domain expertise and lacks cross-platform scalability. To address these limitations, recent studies have explored automated representation learning directly from raw byte sequences[2][7], binary-to-image representations[8], or NLP-inspired techniques[9]. Given that every executable file ultimately consists of raw bytes, byte-level modeling eliminates the need for disassembly and manual feature construction. Notably, end-to-end CNN-based architectures, such as MalConv[2] and MalConv (Non-Neg)[7], have demonstrated the feasibility of raw-binary malware detectors. However, CNNs inherently struggle to model long-range dependencies, which limits their capacity to fully capture non-contiguous control-flow structures and global semantic correlations across malware binaries.

The Transformer architecture, built on multi-head self-attention, has enabled recent advances in artificial intelligence, most visibly in large language models (LLMs)[10][11].

[1] Graduate School of Engineering, Kobe University
[a)] thein.thin@gsuite.kobe-u.ac.jp
[b)] zenmei@port.kobe-u.ac.jp
[*1] https://www.virustotal.com/

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

Its efficacy in modeling both localized and long-range contextual dependencies in sequential data has led to successful applications in diverse domains[10][12][13][14][15]. Although Transformers have demonstrated their efficacy in reverse-engineering tasks such as binary similarity analysis[16], training them directly on whole raw binaries for Windows malware detection remains underexplored[17]. To address this gap, this study examines whether Transformer models trained directly on raw executable binaries can learn a deeper representation of malicious behaviors. Learning directly from raw binary data reduces reliance on domain-specific feature engineering and enhances cross-platform adaptability[2][13].

The main contributions of this study are summarized as follows:

- We propose a byte-level Transformer architecture that operates directly on raw Windows PE binaries without disassembly or manual feature extraction.
- We present a hierarchical Transformer model designed to process binaries up to 1 MB. Our architecture combines three existing techniques: local window attention, periodic global attention, and token merging. In addition, we use gated convolutional downsampling tailored to the structure of executable byte sequences. Together, these mechanisms allow the model to process long byte sequences efficiently.
- We construct a dataset comprising 176,079 PE samples from public repositories, with labels verified using VirusTotal.
- We empirically demonstrate that the proposed model achieves detection performance competitive with MalConv while exhibiting greater robustness to previously unseen obfuscated malware variants.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the details of the proposed method. Section 4 describes the dataset and reports the experimental results and discussion. Section 5 concludes the paper and discusses future work.

2. Related Work

Malware analysis is a systematic process of examining a file's structure, known signatures, and behavioral characteristics to determine the adversary's objective. This process is essential for building effective malware detectors and mitigation strategies. Generally, malware analysis is categorized into static, dynamic, and hybrid approaches. Each approach has its own merits and limitations, and ongoing research and development continuously improves detection techniques to counter evolving malware[4][17].

2.1 Machine Learning for Static Malware Detection

ML-based static malware detectors classify samples as benign or malicious without executing a binary, whereas dynamic detectors execute a binary in a controlled sandbox to observe its runtime behavior. Although both approaches have trade-offs, static analysis plays a critical defense role since malware should ideally be detected before execution. According to the literature[2][8][18], input representations for ML-based malware detectors fall into three main categories: feature-based, image-based, and raw byte representations.

Feature Extraction – The feature-based approach is a widely used strategy for PE malware detection, wherein discriminative features are manually extracted from executable files prior to classification. The EMBER dataset[3] serves as the state-of-the-art benchmark for this approach. It is a labeled dataset with a set of features derived from PE files, such as file header, imported functions, section information, and byte histogram. Using these extracted features, gradient boosting models such as LightGBM achieve strong detection performance. Although this approach remains an important baseline, its effectiveness relies heavily on high-quality domain-specific feature engineering.

Image Representation – Executable binaries are preprocessed into RGB or grayscale images, upon which DNN or ViT models are applied to detect malware. Malware visualization techniques can be traced back as early as 2011[18]. More recently, Seneviratne et al.[8] introduced a self-supervised ViT-based Android malware detection method on a dataset of 1.2 million Android apps, which indicated a performance improvement compared to conventional DNN architectures. Likewise, Jeong et al.[19] employed an enhanced ViT architecture for Windows and Android malware classification, which demonstrated improved robustness against obfuscated samples compared with other ViT-based detectors. However, image-based approaches are constrained by two key limitations: potential loss of fine-grained byte-level details and the additional computational overhead due to image conversion.

Raw Binaries – A raw-binary malware detector operates directly on raw byte sequences, without intermediate representations or feature extraction. MalConv by Raff et al.[2] is the state-of-the-art baseline for end-to-end raw-binary malware detection. MalConv demonstrated that CNNs can learn discriminative features directly from long byte sequences. Subsequent studies have introduced enhancements, particularly focusing on improving robustness against adversarial attacks[7][20] while maintaining comparable detection performance on unseen samples. Because executable files often contain millions of bytes, CNN-based models struggle to capture long-range contextual relationships across the full binary sequence. To address this limitation, this study adopts a multi-head self-attention Transformer architecture[11][13], which is designed to capture global contextual dependencies across input byte sequences.

2.2 Transformer for Malware Detection

Deep neural networks, particularly convolutional neural networks (CNNs) and recurrent neural networks (RNNs), have been widely used for sequence modeling. However, these architectures struggle to capture long-range dependen-

© 2026 Information Processing Society of Japan

cies when processing long sequences[11]. The Transformer architecture introduced the self-attention mechanism, which computes the relevance of each token relative to all other tokens in a sequence. This enables the model to capture both short-range and long-range contextual relationships with better generalization ability[11][21]. While the original Transformer uses an encoder-decoder architecture for machine translation, subsequent studies adapted these components independently for specific tasks. For instance, BERT[22] uses the encoder for bidirectional contextual representations, while GPT[23] relies on the decoder for text generation. Beyond NLP, Transformers have been applied across diverse domains such as computer vision, speech recognition, and cybersecurity, given their ability to process input modalities such as text, images, source code, graphs, and raw byte sequences[12][13][14][15].

The Transformer's capacity to model long-range dependencies makes it well-suited for raw-binary malware detection. Executable files contain millions of bytes, and malicious payloads are often distributed across non-contiguous regions to evade localized signature checks. Although recent studies have applied Transformers to cybersecurity tasks such as binary code similarity, vulnerability detection, and image-based malware classification[8][14][16], these approaches operate on disassembled code or transformed representations, and end-to-end byte-level PE malware detection using Transformers remains largely unexplored.

The attention mechanism of the proposed architecture is built on established Transformer designs. The combination of local window attention and periodic global attention was explored in Longformer[24] and Big Bird[25] to reduce the quadratic cost of self-attention over long sequences. Hierarchical downsampling via token merging was introduced by Bolya et al.[26]. The proposed architecture does not claim novelty in these individual mechanisms; rather, its contribution lies in adapting and combining them specifically for raw PE binaries, alongside gated convolutional downsampling tuned to executable byte structure.

Most closely related to this work is ByteFormer[13], a Transformer architecture designed to operate directly on raw file bytes across arbitrary modalities, including images and audio, without modality-specific preprocessing. While ByteFormer demonstrates that a general-purpose byte-level Transformer can achieve competitive performance across multiple input types, it is not designed to address the specific structural properties of executable binaries such as extreme sequence lengths, sparse informative regions, and the need for aggressive, malware-aware downsampling. The proposed architecture incorporates PE-specific design choices, including gated convolutional downsampling inspired by MalConv[2] and the hierarchical local-global attention stack described above, distinguishing it from ByteFormer.

2.3 Availability of Malware and Benign Executable Samples

Data availability is a fundamental requirement for developing reliable machine learning models. Because PE malware represents one of the most prevalent types of malicious software[1], this study focuses specifically on PE binaries. However, obtaining benchmark datasets for raw-binary malware analysis remains challenging. In particular, access to benign binaries is considerably more limited than access to malware binaries. While public repositories such as VirusShare[*2] and MalwareBazaar[*3] provide large collections of malicious samples, copyright restrictions prevent previous studies from releasing raw benign executables. Instead, these studies offer access to extracted feature vectors[3]. As a result, prior studies have often relied on private or enterprise datasets that are not publicly accessible. To address this limitation, this study constructs a dataset using publicly accessible benign binaries [20][27][28] and those collected from the Windows operating system, with all samples verified via VirusTotal to ensure the absence of malicious behaviors.

3. Proposed Method

This section details the proposed byte-level Transformer for malware detection, referred to as PEFormer. The model operates directly on raw executable bytes without requiring intermediate feature representations. The input to the proposed model is a sequence of executable bytes in the range [0, 255]. First, the embedding layer maps each byte value to a fixed-length feature vector of dimension d. Since byte sequences can be extremely long (up to 2^{20} bytes), processing the entire byte sequence imposes significant GPU memory overhead. To mitigate this challenge, we downsample the sequence with a gated convolutional layer using large filters and strides, inspired by MalConv[2]. The downsampled sequence is then processed by a stack of hierarchical Transformer blocks. The proposed model architecture is illustrated in Fig.1.

3.1 Byte-level Transformer for PE Binaries

A significant challenge in building a byte-level malware detection model is the computational bottleneck associated with processing long byte sequences. Since the standard self-attention in the Transformer scales quadratically with sequence length, processing raw byte sequences directly is computationally expensive. The proposed model addresses this issue through several design choices that reduce memory and compute costs while preserving model performance. First, byte sequences are mapped to embedding vectors and passed through a gated convolutional layer. The input byte sequences are aggressively downsampled via strided convolution to reduce the computational burden prior to attention computation. The gated convolution ensures that contextually important signatures are preserved and emphasized rather than lost in this process.

Next, the latent representations pass through a stack of Transformer blocks designed to capture both localized pat-

[*2] https://virusshare.com/
[*3] https://bazaar.abuse.ch/

© 2026 Information Processing Society of Japan

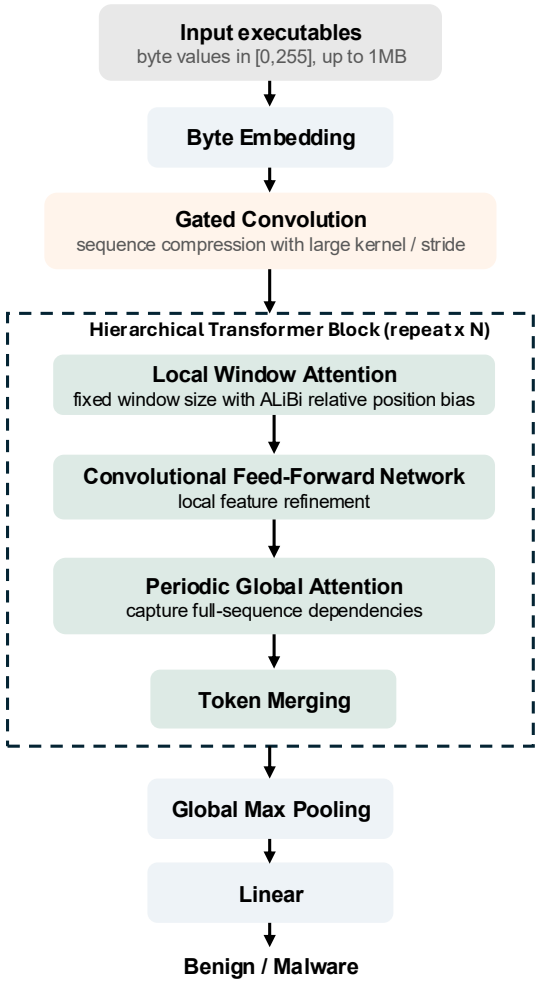


Fig. 1 Overview of the proposed byte-level Transformer. Raw executable bytes are embedded and downsampled via a gated convolution before entering a stack of N hierarchical Transformer blocks. The final representation is pooled and passed to a linear classification layer to produce a benign/malware prediction.

terns and global context. Each block consists of local window attention, a convolutional feed-forward network, periodic global attention, and token merging. The local window attention restricts attention computation to small and non-overlapping windows of a fixed size W, avoiding the quadratic computational complexity required by full self-attention. Within each window, scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B_{\text{ALiBi}}\right)V,$$

where d_k is the per-head dimension and B_{ALiBi} denotes the ALiBi bias. This computation reduces the per-layer attention complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \cdot W)$, as each token only attends to its W local neighbors.

Instead of using standard absolute positional embeddings, which incur memory overhead and fail to generalize beyond fixed sequence boundaries, our model utilizes Attention with Linear Biases (ALiBi)[29]. ALiBi encodes relative positional information directly into the attention matrix by applying a static penalty proportional to token distance. This eliminates the need for learned position embeddings. ALiBi provides strong sequence-length extrapolation capabilities, which are desirable when dealing with variable-length binaries.

Furthermore, to manage long-range dependencies that local window attention cannot capture, global attention blocks are periodically integrated, which enable full-sequence token interactions at a reduced computational overhead. Additionally, the token merging layer performs hierarchical downsampling by concatenating adjacent token pairs and projecting them back to the original model dimension d. Following the Transformer blocks, the variable-length sequence is aggregated via global max pooling to generate a fixed-size representation. Finally, a two-layer MLP classification head maps the pooled representation to class logits. The proposed architecture enables the model to efficiently capture both fine-grained byte patterns and global structural dependencies, making it well-suited for malware detection on raw binaries.

Table 1 Performance comparison of different malware detection approaches.

Model	Accuracy	Precision	Recall	F1
MalConv	0.9805	0.9791	0.9844	0.9818
Transformer+	0.9786	0.9822	0.9776	0.9799
PEFormer	**0.9856**	**0.9874**	**0.9857**	**0.9865**

3.2 Efficient GPU Computation

There are two main concerns regarding input processing: class imbalance and variable-length sequences. The proposed model is designed to process up to 1 MB of input data. However, padding all executable files to a uniform maximum length is computationally inefficient. To address class imbalance and improve training efficiency, we incorporate a weighted, length-aware batch sampling strategy. Class weights are computed using inverse class frequencies, assigning higher sampling probabilities to minority classes. To reduce intra-batch sequence-length variance, dataset indices are sorted by sequence length and partitioned into fixed-size buckets. Samples within each bucket are randomly shuffled to preserve stochastic diversity while ensuring sequences in any given batch are of similar length. This strategy minimizes the number of padding tokens and reduces the computational overhead associated with excessively long sequences. Together, these mechanisms enable scalable and memory-efficient training of a byte-level Transformer for malware detection.

4. Evaluation

We collected benign and malicious PE executables from Practical Security Analytics[27], with additional benign samples from Lucas et al.[20] and Saha et al.[28]. All executables were verified using VirusTotal, and duplicate samples were removed. If a benign sample is flagged as malicious by one or more detectors on VirusTotal, it is excluded from the benign dataset. Moreover, a sample is labeled ma-

© 2026 Information Processing Society of Japan

Table 2 Hyperparameters used in the proposed model.

Hyperparameter	Value
Batch size	32
Number of heads	8
Learning rate	1×10^{-4}
Depth	2
Weight decay	1×10^{-3}
Dropout	0.05
Vocabulary size	256
Convolution kernel	400
Transformer model dimension	200
Convolution stride	400
Window size	256
Byte embedding dimension	16

licious if it is flagged by at least five VirusTotal detection engines, consistent with common practice for reducing label noise from individual scanners. The final dataset contains 176,079 samples: 81,956 benign and 94,123 malicious. Samples exceeding the 1 MB input limit were excluded from the dataset rather than truncated. The dataset is divided into training (65%), validation (15%), and test (20%) sets.

Two baseline models are selected for comparison, as shown in Table 1. MalConv[2] processes raw bytes directly and serves as a widely used benchmark for byte-level malware detection. Transformer+ is a baseline Transformer architecture loosely based on ByteFormer[13], which uses a Transformer-based backbone operating directly on raw file bytes but was originally designed for multimedia classification rather than malware detection. Specifically, Transformer+ consists of a byte embedding layer, convolutional downsampling, a stack of Transformer blocks with windowed attention and standard positional encoding, and a final classification head identical in structure to that of the proposed model. We include Transformer+ as a useful architectural reference point. We used the same hyperparameters as MalConv[*4], except that we set $batch_size = 32$. We trained all models for 30 epochs. Table 2 lists the hyperparameters used to train the proposed model. Table 3 compares model size and computational cost across the three models. Although MalConv has the fewest parameters, it requires longer training time per epoch than Transformer+ because it processes every sample padded to the full 1 MB input length, whereas the Transformer-based models use the length-aware bucketed batching strategy described in Section 3.2, which substantially reduces padding overhead for the majority of samples shorter than the maximum length.

4.1 Experiment on a Small Set of Unseen Malware Samples and Their Obfuscated Variants

We collected binaries from the Malware Analysis and Obfuscation Dataset (v1.0) on Kaggle[*5]. The original dataset contains 202 malware samples and 6,993 corresponding obfuscated samples. After selecting PE binaries and their obfuscated versions under 1 MB, we obtain 127 original malware samples and 254 obfuscated samples. We used the models reported in Table 1 for this experiment. The final hyperparameter configuration (kernel/stride=400, depth=2) used for the main results in Table 1 was selected using validation-set performance in Section 4.2, prior to and independently of the obfuscation evaluation reported here.The results confirm that detection accuracy on obfuscated samples depends on the trained model's hyperparameters. In most cases, MalConv fails to correctly classify the obfuscated variants, while our proposed model achieves a detection accuracy of 160/254 even in its worst-case hyperparameter configuration reported in Table 4.

The wide performance gap between the best-case detection accuracy (99.2%) and the worst-case (63.0%) on obfuscated samples suggests that robustness to obfuscation is considerably more sensitive to architectural hyperparameters than in-distribution accuracy. We attribute this to the aggressive convolutional downsampling (kernel/stride=500) of the byte sequence before attention is applied, discarding fine-grained byte patterns that obfuscation tools tend to perturb locally while leaving broader structural regions intact. This indicates that while the proposed architecture is broadly robust on standard in-distribution detection, hyperparameter choices that are optimal for clean-sample accuracy do not automatically transfer to robustness against obfuscated variants, and the two objectives may need to be tuned independently.

4.2 Hyperparameter Sensitivity Study

The hyperparameter sensitivity study was conducted to examine the influence of the training hyperparameters on the proposed malware detection model. The evaluated settings include variations in learning rate, weight decay, model dimension, Transformer depth, dropout rate, convolutional kernel size and stride, and other architectural parameters. Overall, the evaluated settings achieve consistently high performance, with a precision of 0.9871 ± 0.0036, a recall of 0.9797 ± 0.0075, and an F1-score of 0.9832 ± 0.0035. This relatively narrow performance range suggests that the proposed architecture is robust to moderate changes in its hyperparameters. Nevertheless, several architectural choices have a noticeable influence on the balance between precision, recall, model complexity, and computational efficiency.

Using a convolutional kernel and stride of 400, combined with a shallow Transformer (depth of 2), appears to provide a favorable trade-off between representation quality and computational efficiency. Using a larger convolutional kernel and stride (500) with Transformer depth of 3 yields the highest trainable parameter count but does not outperform the model using a smaller convolutional kernel and stride (400). This suggests that excessive downsampling may discard useful local byte-level information, whereas a moderately smaller convolutional window can preserve more discriminative information before the Transformer layers process the resulting representations. Reducing the kernel and stride to 400 while also reducing the transformer depth from

[*4] https://github.com/elastic/ember/tree/master/malconv
[*5] https://www.kaggle.com/datasets/khaledwrightedu/malware-analysis-and-obfuscation-dataset-v1-0

© 2026 Information Processing Society of Japan

Table 3 Comparison of model size, training time, and inference time.

Model	Parameters	Training time per epoch	GPU	Avg. inference time per sample
MalConv	1.04M	∼874 s	Google Colab A100 GPU	∼8.6 ms
Transformer+	2.57M	∼485 s	Google Colab A100 GPU	∼9.4 ms
PEFormer	4.29M	∼1128 s	Google Colab A100 GPU	∼18.1 ms

Table 4 Detection accuracy on unseen malware samples and their obfuscated variants.

Method	Malware samples (n=127)	Obfuscated samples (n=254)
MalConv	117/127 (92.1%)	127/254 (50.0%)
Transformer+	105/127 (82.7%)	153/254 (60.2%)
PEFormer (kernel/stride=400, depth=2)	117/127 (92.1%)	**252/254 (99.2%)**
PEFormer (kernel/stride=500, depth=3)	**122/127 (96.1%)**	160/254 (63.0%)

three layers to two shrinks the model by roughly 25% with no corresponding drop in F1-score on the evaluated dataset. This indicates that increasing Transformer depth does not necessarily improve detection performance.

The learning rate and weight decay also affect the precision-recall balance. More conservative optimization settings achieve competitive performance, whereas increasing the learning rate and weight decay simultaneously does not lead to an overall improvement. In particular, the best-performing settings maintain a relatively moderate learning rate and weight decay, suggesting that aggressive optimization and regularization are unnecessary for the proposed architecture and evaluated dataset. The experimental results indicate that the smaller architecture tolerates a higher learning rate well, provided weight decay remains moderate, and that learning rate and weight decay should be tuned jointly rather than in isolation.

A performance drop occurs in the configuration that combines a smaller batch size, a much smaller convolution kernel/stride, and a wider model dimension. Despite its comparatively large trainable parameter count, this configuration produces the lowest F1-score. This drop suggests that an overly small convolutional window fails to aggregate sufficient local byte-level context for reliable malware detection. Taken together, these results indicate that a moderate convolution kernel and stride, a shallower Transformer, and moderate regularization offer the best trade-off between detection performance and model size, motivating the choice of the smaller, shallower configuration as the final model used in the main evaluation.

To assess the contribution of the gating mechanism in the downsampling layer, we conducted an ablation in which the gated convolution was replaced with a standard convolution of identical kernel and stride (400), with all other hyperparameters unchanged. This modification results in a modest decrease in overall detection performance (precision: 0.9860, recall: 0.9826, F1: 0.9843, accuracy: 0.9832) compared to the gated configuration (F1: 0.9865). The effect is considerably more pronounced on the unseen and obfuscated evaluation sets from Section 4.1, where detection accuracy drops from 117/127 to 116/127 on unseen malware and, more notably, from 252/254 to 126/254 on obfuscated variants. This substantial degradation on obfuscated samples suggests that the gating mechanism plays a meaningful role in preserving discriminative byte-level patterns during aggressive downsampling, which is consistent with our hypothesis in Section 3.1 that gated convolution helps retain contextually important signatures that would otherwise be lost.

We also note that the relatively narrow performance range observed across hyperparameter configurations may be partly influenced by the scale of the training dataset (176,079 samples). Architectural factors such as Transformer depth and model dimension typically yield larger performance differences as dataset size increases, since deeper or wider models generally require more training data to realize their full representational capacity. Some of the trends observed here may reflect the current dataset scale rather than an inherent capacity of the architecture. Evaluating the proposed model on larger datasets remains an important direction for future work, both to validate these hyperparameter trends and to further assess generalization. We also plan to conduct a component-wise ablation study to determine the contribution of each architectural component. This would clarify which component is primarily responsible for the model's improved robustness to obfuscated variants observed in Section 4.1 or whether that robustness instead stems from the pipeline as a whole.

4.3 Limitations

This study has several limitations. First, the proposed model is evaluated exclusively on Windows PE binaries, while its applicability to other executable formats (e.g., ELF and Mach-O) remains to be investigated. Consequently, the extent to which the proposed approach generalizes across executable formats remains an open research question. Second, the train/validation/test split (65%/15%/20%) is constructed via random sampling over deduplicated hashes, without enforcing family-disjoint or temporal separation between splits. Polymorphic or closely related variants of the same malware family may appear in both the training and test sets, which could inflate the reported detection performance[30]. However, MalConv, Transformer+, and the proposed PEFormer were all trained and evaluated under identical splitting conditions. Consequently, the relative performance comparison between these architectures remains meaningful. Furthermore, the obfuscation robustness ex-

© 2026 Information Processing Society of Japan

periment described in Section 4.1 is evaluated on an entirely separate, externally sourced dataset, so it is not subject to this concern.

Third, the model architecture imposes a fixed input-size limit of 1 MB. Consequently, malware payloads embedded within binaries exceeding this limit, as well as malicious behaviors introduced through post-execution downloads, remain beyond the scope of the static analysis considered in this study, as such samples were excluded from both training and evaluation. One potential approach for handling larger binaries is to partition them into smaller segments and perform detection on each segment. However, the effectiveness and reliability of such a segmentation-based strategy have yet to be systematically examined.

Fourth, the obfuscation robustness experiment (Section 4.1) is conducted on a relatively small evaluation set comprising 127 original malware and 254 obfuscated samples. Although the results demonstrate promising robustness against obfuscation, broader evaluation involving a more diverse range of obfuscation techniques, tools, and malware families would provide stronger evidence regarding the generalizability of the findings. Finally, as a purely static detector, the proposed model inherits the limitation of static analysis approaches: malicious behaviors that manifest exclusively at runtime, such as dynamic unpacking or staged payload delivery, cannot be directly captured from the raw byte sequence alone.

5. Conclusion

This paper investigated whether a byte-level Transformer architecture, applied directly to raw Windows PE binaries, can learn richer representations of malicious behaviors than conventional CNN-based detectors. To make self-attention computationally efficient on binaries up to 1 MB, the proposed architecture combines gated convolutional downsampling with a hierarchical Transformer design. On a held-out set of unseen obfuscated malware variants, the proposed model substantially outperformed MalConv, correctly classifying up to 99.2% of obfuscated samples in its best hyperparameter configuration compared to MalConv's 50.0%. This suggests that the model's attention-based global context modeling generalizes better to previously unseen obfuscation strategies than the convolution-only design used by MalConv. On our dataset of 176,079 PE samples, PE-Former and MalConv achieved comparable performance (F1: 0.9865 vs. 0.9818), with the proposed model using roughly four times as many parameters. Ablation results further indicated that a moderate convolutional downsampling combined with a shallow Transformer offers the best trade-off between detection performance and model size.

Several directions remain for future work. First, evaluating robustness against adversarial and evasion attacks is crucial, these remain a significant concern for raw-binary detectors. Second, extending the maximum supported input length beyond 1 MB and evaluating on non-Windows binary formats (e.g., ELF, Mach-O) would test the generalizability of the proposed approach across platforms. Finally, incorporating explainability techniques to localize which byte regions drive detection decisions could improve analyst trust and support downstream forensic investigation.

5.1 Dataset Availability

To support reproducibility and to help address the shortage of publicly available benign PE samples noted in Section 2.3, we will release the sample hashes, their collection sources, and associated VirusTotal scan metadata upon publication. Due to copyright and distribution restrictions on the underlying binaries themselves (particularly Windows system files), we are unable to redistribute the raw executables directly, but researchers can reconstruct the dataset using the provided identifiers and the original public sources described in Section 4.

Acknowledgments This work was partially supported by JST K Program Japan Grant Number JPMJKP24K1.

References

[1] K. van Liebergen, J. Caballero, P. Kotzias, C. Gates, A deep dive into the virustotal file feed, in: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Springer, 2023, pp. 155–176.

[2] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, C. Nicholas, Malware detection by eating a whole exe, arXiv preprint arXiv:1710.09435 (2017).

[3] H. S. Anderson, P. Roth, Ember: an open dataset for training static pe malware machine learning models, arXiv preprint arXiv:1804.04637 (2018).

[4] D. Ucci, L. Aniello, R. Baldoni, Survey of machine learning techniques for malware analysis, Computers & Security 81 (2019) 123–147.

[5] Z. Zhang, P. Qi, W. Wang, Dynamic malware analysis with feature engineering and feature learning, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 34, 2020, pp. 1210–1217.

[6] A. Damodaran, F. D. Troia, C. A. Visaggio, T. H. Austin, M. Stamp, A comparison of static, dynamic, and hybrid analysis for malware detection, Journal of Computer Virology and Hacking Techniques 13 (1) (2017) 1–12.

[7] W. Fleshman, E. Raff, J. Sylvester, S. Forsyth, M. McLean, Non-negative networks against adversarial attacks, arXiv preprint arXiv:1806.06108 (2018).

[8] S. Seneviratne, R. Shariffdeen, S. Rasnayaka, N. Kasthurirachchi, Self-supervised vision transformers for malware detection, IEEE access 10 (2022) 103121–103135.

[9] M. Mimura, R. Ito, Applying nlp techniques to malware detection in a practical environment, International Journal of Information Security 21 (2) (2022) 279–291.

[10] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, arXiv preprint arXiv:2303.18223 1 (2) (2023) 1–124.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, et al., Attention is all you need, Advances in neural information processing systems 30 (1) (2017) 5998–6008.

[12] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, M. Shah, Transformers in vision: A survey, ACM computing surveys (CSUR) 54 (10s) (2022) 1–41.

[13] M. Horton, S. Mehta, A. Farhadi, M. Rastegari, Bytes are all you need: Transformers operating directly on file bytes, arXiv preprint arXiv:2306.00238 (2023).

[14] W. Ahmad, S. Chakraborty, B. Ray, K.-W. Chang, A transformer-based approach for source code summarization, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 4998–5007.

[15] A. Gillioz, J. Casas, E. Mugellini, O. Abou Khaled, Overview of the transformer-based models for nlp tasks, in: 2020 15th

© 2026 Information Processing Society of Japan

Conference on computer science and information systems (FedCSIS), IEEE, 2020, pp. 179–183.
[16] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, C. Zhang, Jtrans: Jump-aware transformer for binary code similarity detection, in: Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis, 2022, pp. 1–13.
[17] P. Kunwar, K. Aryal, M. Gupta, M. Abdelsalam, E. Bertino, Sok: Leveraging transformers for malware analysis, IEEE Transactions on Dependable and Secure Computing (2025).
[18] L. Nataraj, S. Karthikeyan, G. Jacob, B. S. Manjunath, Malware images: visualization and automatic classification, in: Proceedings of the 8th international symposium on visualization for cyber security, 2011, pp. 1–7.
[19] I.-W. Jeong, H.-J. Lee, G.-N. Kim, S.-H. Choi, Malformer: a novel vision transformer model for robust malware analysis, IEEE Access (2025).
[20] K. Lucas, M. Sharif, L. Bauer, M. K. Reiter, S. Shintre, Malware makeover: Breaking ml-based static analysis by modifying executable bytes, in: Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security, 2021, pp. 744–758.
[21] Y. Bai, J. Mei, A. L. Yuille, C. Xie, Are transformers more robust than cnns?, Advances in neural information processing systems 34 (2021) 26831–26843.
[22] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), 2019, pp. 4171–4186.
[23] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training (2018).
[24] I. Beltagy, M. E. Peters, A. Cohan, Longformer: The long-document transformer, arXiv preprint arXiv:2004.05150 (2020).
[25] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, et al., Big bird: Transformers for longer sequences, Advances in neural information processing systems 33 (2020) 17283–17297.
[26] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, J. Hoffman, Token merging: Your vit but faster, arXiv preprint arXiv:2210.09461 (2022).
[27] PE Malware Machine Learning Dataset (2021), `https://practicalsecurityanalytics.com/pe-malware-machine-learning-dataset/`, Accessed on August 2025.
[28] S. Saha, W. Wang, Y. Kaya, S. Feizi, T. Dumitras, Drsm: de-randomized smoothing on malware classifier providing certified robustness, in: International Conference on Learning Representations, Vol. 2024, 2024, pp. 47666–47686.
[29] O. Press, N. A. Smith, M. Lewis, Train short, test long: Attention with linear biases enables input length extrapolation, arXiv preprint arXiv:2108.12409 (2021).
[30] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, L. Cavallaro, {TESSERACT}: Eliminating experimental bias in malware classification across space and time, in: 28th USENIX security symposium (USENIX Security 19), 2019, pp. 729–746.

© 2026 Information Processing Society of Japan